Resilience-Alpha — Documentation complète

*Tout ce qui se passe dans le projet, de la requête d'un utilisateur à la reprise automatique après une attaque. Plateforme de paiement **fictive**, réseau **fermé**, données **fictives**.*

---

1. Vue d'ensemble

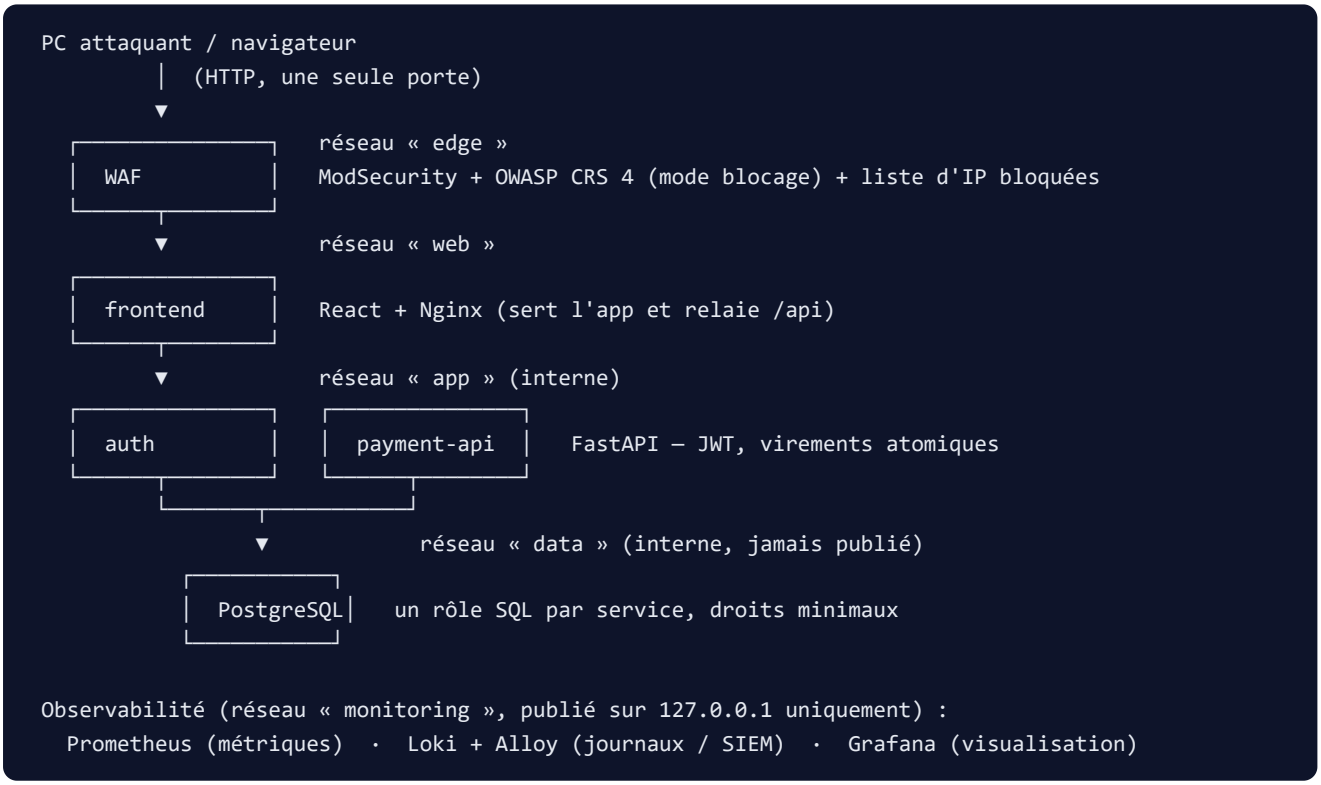
Resilience-Alpha simule une petite banque en ligne qui **se défend et se répare toute seule**. Le principe tient en une phrase :

> On **mesure tout** en continu → un modèle **détecte** si c'est une attaque → un assistant **explique** → un moteur **agit** et **vérifie** que le service est rétabli — le tout **sans intervention humaine** (*Zero-Touch Recovery*).

Trois rôles sont **volontairement séparés** (exigence du sujet) : **la détection décide, le RAG explique, la remédiation agit.**

---

2. Architecture : les briques et qui parle à qui



Service	Rôle	Exposition
waf	Seul point d'entrée. Filtre les attaques web (CRS 4), bloque les IP listées, journalise en JSON	`127.0.0.1:8088` (ou IP réseau pour la démo)
frontend	Application web + proxy vers les API	interne
auth	Connexion, jetons JWT, verrouillage après 10 échecs	interne (métriques `:9000`)
payment-api	Comptes et virements fictifs, atomiques et idempotents	interne (métriques `:9000`)
db	PostgreSQL ; un rôle par service, droits minimaux	interne, **jamais publié**
prometheus	Collecte les métriques + calcule les **KPI** (une seule source de vérité)	`127.0.0.1:9090`
loki + alloy	Centralise les journaux (SIEM), permet de retrouver l'IP d'un attaquant	interne
grafana	Tableaux de bord	`127.0.0.1:3000`

Et, **hors conteneurs**, les outils Python du projet : - **console** — les deux interfaces web (SOC + attaquant) · `127.0.0.1:8060` - **détection** (kNN) · **assistant RAG** (Ollama) · **moteur de remédiation** - **harnais d'attaque** (`labtest/`) pour jouer les attaques.

---

3. Ce qui se passe pour une requête NORMALE

1. L'utilisateur se connecte sur `http://127.0.0.1:8088` → la requête entre par le **WAF**.
2. Le WAF inspecte (règles OWASP), puis transmet au **frontend** qui relaie vers **auth**.
3. `auth` vérifie le mot de passe (haché en Argon2id) et renvoie un **JWT**.
4. L'utilisateur fait un virement → `payment-api` vérifie le jeton, exécute la transaction en base de façon **atomique** (débit + crédit ensemble, ou rien).
5. À chaque étape, chaque service écrit une **ligne de journal JSON** (vers Loki) et expose des **métriques** (vers Prometheus).

C'est cette activité normale qui sert de **référence** : le détecteur sait à quoi ressemble « tout va bien ».

---

4. Ce qui se passe pendant une ATTAQUE (le cycle complet)

Étape 1 — L'attaque

Depuis le PC attaquant (ou la page d'attaque locale), on joue un profil : **DDoS**, **force brute**, **injection SQL**, **scan de ports**, ou une **panne de service**. Tout vise uniquement notre hôte, sur le réseau fermé.

Étape 2 — La collecte (en continu)

Prometheus voit les KPI bouger (requêtes/s, erreurs, latence, connexions, requêtes bloquées par le WAF, échecs de connexion…). Loki enregistre les lignes de journal correspondantes.

Étape 3 — La détection (décide)

Un modèle **kNN** (plus proches voisins) compare l'état courant (un vecteur de ~18 KPI) aux profils connus. Il renvoie un **verdict** : l'étiquette (`ddos` , `brute_force` , `sql_injection` , `port_scan` , `service_down` , `normal`), une **confiance**, et les KPI qui ont déclenché. Si l'état est trop inhabituel, il répond `inconnu` → escalade humaine.

Étape 4 — L'explication (RAG / LLM — explique seulement)

L'assistant (Ollama en local, modèle ≤ 8 B) lit une **base de connaissance** sur les attaques (fichiers indexés dans FAISS) et rédige, en langage clair, **ce qui se passe** et **quoi faire**. Il n'exécute **aucune** action.

Étape 5 — La remédiation (agit, puis vérifie)

Le moteur confirme d'abord l'attaque avec un **anti-rebond** (plusieurs détections d'affilée), puis :

Type	Nature	Action
Panne de service	disponibilité	**redémarre** le conteneur, **vérifie** la reprise
DDoS	disponibilité	bloque / stoppe la source, redémarre l'API si besoin, vérifie
Force brute	abus	bloque la source (verrouillage de compte déjà actif), vérifie le métier
Scan de ports	reconnaissance	bloque la source, surveille
Injection SQL	compromission	**confinement seul** — bloque, alerte, **jamais « résolu »** sans humain

Comment l'attaque est réellement stoppée : - En **production** : l'IP de l'attaquant est **bloquée au WAF** (ses requêtes passent en 403). L'IP est retrouvée dans **Loki** (journaux WAF/auth). - En **labo fermé** : l'attaque vient de notre propre harnais → son IP est la passerelle Docker locale, **protégée** (on ne bloque jamais notre infra). La remédiation **coupe alors l'attaque à la source** (étape `lab-containment`). C'est pourquoi, en démo, l'attaque **s'arrête vraiment**.

Étape 6 — La vérification (preuve de reprise)

On ne se contente pas d'un redémarrage : le moteur refait un `/health` **et** une **transaction de test** (le compte technique `probe` vire 0,01 €). Si elle réussit, l'incident est `resolved` ; sinon `failed` → intervention humaine.

Étape 7 — Le journal

Chaque étape (détection → confinement → redémarrage → vérification → clôture) est écrite dans un **journal JSONL** et affichée en direct sur le tableau de bord SOC.

Garde-fous : actions **idempotentes** (IP déjà bloquée = rien, service déjà sain = pas de redémarrage), **délai de repos** entre deux actions du même type, **confiance minimale** requise.

---

5. Les deux interfaces

- **Page SOC** (`http://127.0.0.1:8060`) — bandeau *normal / attaque / inconnu*, tuiles de KPI, courbe de trafic, explication du RAG, **remédiation recommandée** et **journal de remédiation** en direct.
- **Page attaquant** (`http://127.0.0.1:8060/attacker`) — boutons pour lancer / arrêter chaque attaque (démo sur un seul PC). Depuis un **vrai 2e PC**, on utilise le harnais en ligne de commande

(voir `redteam/PC-ATTAQUANT.md`).

---

6. Sécurité : pourquoi la défense n'est pas elle-même une faille

- Données **100 % fictives** ; **aucun secret en clair** (secrets séparés, versions chiffrées).
- WAF en **mode blocage** (OWASP CRS 4) ; audit **sans** mots de passe ni jetons dans les journaux.
- L'IP client vient **du WAF seul** → pas d'usurpation par en-tête HTTP.
- Conteneurs **non-root**, système de fichiers **en lecture seule**, capacités retirées, `no-new-privileges` ; **réseaux internes** isolés ; seul le WAF est publié.
- **Rôles SQL séparés** : `payment_svc` ne peut pas lire la table des utilisateurs ni supprimer une transaction.
- **Remédiation à droits minimaux** : seulement des actions prédéfinies sur des cibles connues (IP validée et hors liste « jamais bloquée », redémarrage limité à une **liste blanche**), **jamais de commande arbitraire**.
- Les attaques restent **sur le réseau fermé de l'équipe**, jamais sur le campus.

---

7. Comment lancer le projet

Prérequis : **Docker Desktop (WSL2)**, **Python 3.12**, (et **Ollama** pour l'explication RAG).

```
# 1. secrets + plateforme
python deploy/gen_secrets.py
docker compose -f deploy/docker-compose.yml up -d --build
python tests/smoke_test.py            # vérifie que tout répond

# 2. supervision + interfaces web
python -m console.app                 # http://127.0.0.1:8060  et  /attacker

# 3. remédiation automatique (le pont détection -> action)
python -m remediation.run

# 4. provoquer un incident (démo Zero-Touch)
docker compose -f deploy/docker-compose.yml stop db       # panne -> réparée toute seule
#   ou une attaque depuis la page /attacker, ou depuis le PC attaquant
```

Interface	Adresse
Application	http://127.0.0.1:8088
Console SOC / attaquant	http://127.0.0.1:8060 · http://127.0.0.1:8060/attacker
Grafana	http://127.0.0.1:3000
Prometheus	http://127.0.0.1:9090

---

8. Résultats validés

- **Panne de base de données** : arrêtée → détectée → **conteneur redémarré → transaction de test réussie →** `resolved` **en ~14 s**, entièrement automatique.
- **Injection SQL** : IP source identifiée via Loki → **confinement** (pas de redémarrage, analyse humaine requise) — disponibilité et compromission bien distinguées.
- **Blocage / arrêt de source** : idempotent ; ne bloque jamais sa propre passerelle.
- **Tests** : 28 tests unitaires passent.

---

9. Limites connues

- **Machine à 2 cœurs** : les attaques sont volontairement **modérées** (produire la *signature*, pas écrouler l'hôte). Un flot trop fort sature la VM Docker Desktop.
- La détection des attaques **réseau** dépend du modèle entraîné : un **ré-entraînement** sur les attaques douces fiabilise les verdicts. La reprise sur **panne** fonctionne indépendamment.
- Mesure des ressources **par conteneur** limitée sous Docker Desktop (stockage containerd) : on utilise les métriques de processus.

---

10. Où regarder dans le code

Dossier	Contenu
`deploy/`	`docker-compose.yml`, génération des secrets
`services/`	`auth`, `payment-api` (FastAPI), `waf` (règles ModSecurity)
`observability/`	configuration Prometheus (KPI), Loki, Grafana
`data/`	collecteur de données + trafic normal de référence
`detection/`	modèle kNN, caractéristiques (KPI), construction de l'index
`assistant/`	RAG (Ollama + FAISS) + base de connaissance des attaques
`remediation/`	moteur, actions, vérification, journal
`console/`	backend + les deux pages web
`labtest/` · `redteam/`	harnais d'attaque + guides
``	